Computational Methods on Black Hole Imaging

Relativistic Ray Tracing via Automatic Differentiation and Geometric Integration

Christos Mermigkas

Aristotle University of Thessaloniki
Computational Physics MSc

October 2026

Outline

1. Motivation
2. Computational Methods
   - The Geodesic Equation
   - Method 1: Automatic Differentiation
   - Method 2: Manifold Projection
3. Validation
4. Imaging Methodology
5. Results: Imaging
6. Conclusions

From Gargantua to M87*: The Science of Black Hole Imaging

- **2014:** The movie *Interstellar* showed the world a realistic black hole "Gargantua" - computed using actual General Relativity equations.
- **2019:** The Event Horizon Telescope (EHT) revealed the first real image of a black hole (M87*).
- **The common thread:** Both rely on **relativistic ray tracing** - tracing light paths through curved spacetime.

The Central Question

What would an observer actually see when looking at a black hole?

Answering this requires solving the geodesic equation with **machine-precision accuracy** in a chaotic, strong-field environment.

The Computational Challenge

- **Goal:** Trace millions of photon trajectories backwards from the observer to the source.
- **Problem:** Near the photon sphere, trajectories are **chaotic** (Lyapunov instability).
- Tiny numerical errors $\rightarrow$ exponential divergence $\rightarrow$ unphysical results.
- **Requirement:** Machine-precision accuracy over long integration times.

Our Solution

A metric-agnostic framework in Julia combining:

1. Automatic Differentiation
2. Manifold Projection

What We Compute: The Geodesic Equation

- **Physical problem:** Find the path $x^\mu(\lambda)$ of a photon in curved spacetime.
- **The equation:**

$$\ddot{x}^\mu + \Gamma^\mu{}_{\alpha\beta}\dot{x}^\alpha\dot{x}^\beta = 0 \tag{1}$$

- where $\Gamma^\mu{}_{\alpha\beta}$ are the Christoffel symbols:

$$\Gamma^\mu{}_{\alpha\beta} = \frac{1}{2}g^{\mu\sigma}\left(\partial_\alpha g_{\beta\sigma} + \partial_\beta g_{\alpha\sigma} - \partial_\sigma g_{\alpha\beta}\right) \tag{2}$$

- **The bottleneck:** We need $\partial_\sigma g_{\mu\nu}$ - the derivatives of the metric tensor.

AD: Why Finite Differences Fail

- **Traditional approach:** Finite Differences (FD)

$$\partial_\sigma g_{\mu\nu} \approx \frac{g_{\mu\nu}(x+h) - g_{\mu\nu}(x-h)}{2h} \tag{3}$$

- **Two competing errors:**
  - **Truncation error:** $\mathcal{O}(h^2)$ - decreases as $h \to 0$
  - **Round-off error:** $\mathcal{O}(\epsilon_{\text{mach}}/h)$ - increases as $h \to 0$
- **Result:** U-shaped error curve.

AD: How Dual Numbers Work

- **Idea:** Extend real numbers with an infinitesimal ϵ where $\epsilon^2 = 0$.
- **Dual number:** $x = a + b\epsilon$ (value a, derivative b).
- **Key insight:** If we evaluate $f(x + 1 \cdot \epsilon)$, the Taylor expansion gives:

$$f(x+\epsilon) = f(x) + f'(x) \underbrace{\epsilon}_{\text{derivative part}} + \frac{1}{2} f''(x) \underbrace{\epsilon^2}_{=0} + \cdots = f(x) + f'(x)\epsilon \tag{4}$$

- **How it works in practice:**
  1. Start with input $x = a + 1 \cdot \epsilon$ (value a, derivative 1).
  2. Apply operations (add, multiply, sin, exp, etc.) using dual number rules.
  3. Each operation propagates both value and derivative.
  4. Final result: $f(a) + f'(a)\epsilon$ — the coefficient of ϵ is the exact derivative!
- **Chain rule:** Applied algebraically to every operation:
  - $(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon$
  - $(a + b\epsilon) \cdot (c + d\epsilon) = ac + (ad + bc)\epsilon$
- **Result: Exact** derivatives to machine precision ($\sim 10^{-16}$).

AD: Example with Chain Rule — $f(x) = x \cdot \sin(x)$ at $x = \pi/2$

- **Input:** $x = \frac{\pi}{2} + 1 \cdot \epsilon$.
- **Step 1:** Compute $\sin(x)$ using dual sine rule $\sin(a + b\epsilon) = \sin(a) + \cos(a)\, b\epsilon$:

$$\sin\left(\tfrac{\pi}{2} + \epsilon\right) = \sin\left(\tfrac{\pi}{2}\right) + \cos\left(\tfrac{\pi}{2}\right) \epsilon = 1 + 0 \cdot \epsilon = 1 \tag{5}$$

- **Step 2:** Multiply $x \cdot \sin(x)$:

$$\begin{aligned} \left(\tfrac{\pi}{2} + 1\epsilon\right) \cdot (1 + 0\epsilon) &= \tfrac{\pi}{2} \cdot 1 + \left(\tfrac{\pi}{2} \cdot 0 + 1 \cdot 1\right) \epsilon \\ &= \tfrac{\pi}{2} + 1 \cdot \epsilon \end{aligned}$$

- **Read off:** $f(\pi/2) = \pi/2, \quad f'(\pi/2) = 1 \quad \checkmark$

Key takeaway

Every elementary function (sin, cos, exp, log, ...) has a dual-number extension. By chaining them, the computer builds the full derivative of *any* composite function - exactly, with no truncation error.

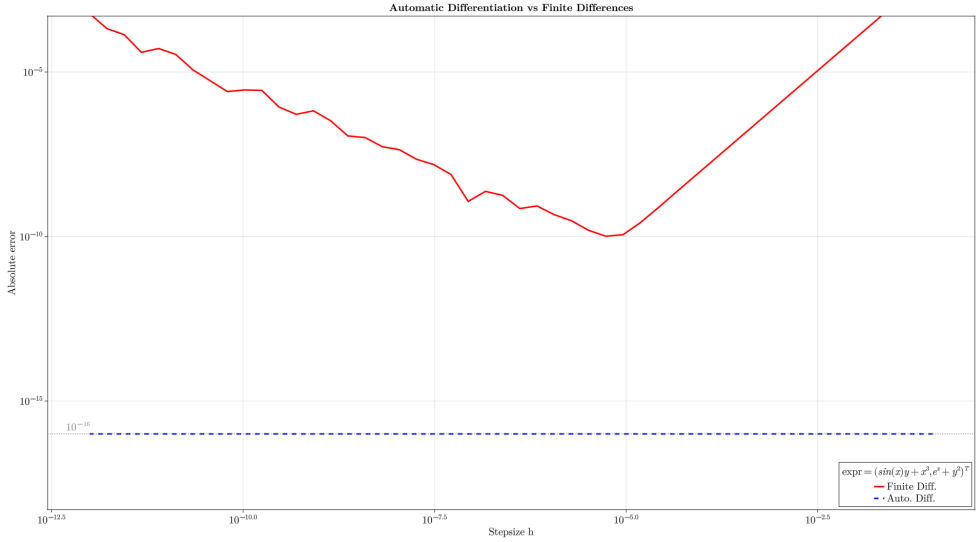
AD: What, Why, How

- **What we compute:** $\partial_\sigma g_{\mu\nu}$ and $\Gamma^{\mu}{}_{\alpha\beta}$ automatically.
- **Why:** To feed the geodesic equation with exact accelerations.
- **How:** Julia's ForwardDiff package propagates dual numbers through the metric function.

Bonus!

Metric-agnostic, works for any spacetime without manual derivation.

AD: Test Result — FD vs AD



The Problem: Invariant Drift

- **What happens:** Standard Runge-Kutta accumulates floating-point errors.
- **Physical constraints (the manifold $\mathcal{N}$):**
  1. **Null condition:** $g_1(\boldsymbol{y}) \equiv g_{\mu\nu} u^\mu u^\nu = 0$
  2. **Energy:** $g_2(\boldsymbol{y}) \equiv E - E_0 = 0$
  3. **Angular momentum:** $g_3(\boldsymbol{y}) \equiv L_z - L_{z0} = 0$
- **Consequence:** The numerical solution $\tilde{\boldsymbol{y}}$ drifts off $\mathcal{N}$.
- Near the photon sphere $\rightarrow$ exponential divergence.

Manifold Projection: The Mathematical Idea

- **Goal:** After each RK step, project $\tilde{\boldsymbol{y}}$ back onto $\mathcal{N}$.
- **Method:** Lagrange multipliers - minimize distance subject to constraints:

$$\mathcal{S}(\boldsymbol{y}^*, \boldsymbol{\lambda}) = \frac{1}{2}\|\boldsymbol{y}^* - \tilde{\boldsymbol{y}}\|^2 + \boldsymbol{\lambda}^T \boldsymbol{g}(\boldsymbol{y}^*) \tag{6}$$

- **Solution:**

$$\boldsymbol{y}^* = \tilde{\boldsymbol{y}} - \boldsymbol{J}^T \boldsymbol{\lambda}, \quad \boldsymbol{J} = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{y}} \tag{7}$$

- **Key property:** Correction is ***orthogonal*** to the physical flow - no artificial damping.

Manifold Projection: Newton-Raphson Solver

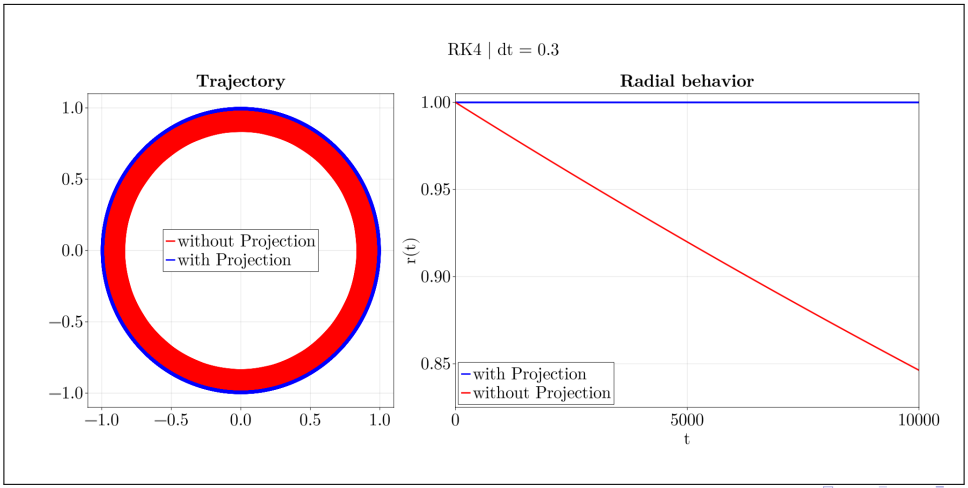
- **We need to solve:** $\boldsymbol{g}(\boldsymbol{y}^*) = 0$ for $\boldsymbol{\lambda}$.
- **Newton-Raphson iteration:**

$$\boldsymbol{G}\,\Delta\boldsymbol{\lambda}^{(k)} = -\boldsymbol{g}(\boldsymbol{y}^{(k)}), \quad \boldsymbol{G} = \boldsymbol{J}\boldsymbol{J}^T \tag{8}$$

- **Algorithm:**
  1. Initialize $\boldsymbol{\lambda}^{(0)} = 0$, $\boldsymbol{y}^{(0)} = \tilde{\boldsymbol{y}}$
  2. Compute $\boldsymbol{J}$ using AD (exact Jacobian!)
  3. Iterate until $\|\Delta\boldsymbol{\lambda}\| \leq tol$
- **Quadratic convergence:** Correct digits double each iteration.

Manifold Projection: Interactive Demo

- **Test:** Harmonic oscillator $\dot{x} = y$, $\dot{y} = -x$ (analytical: circle).

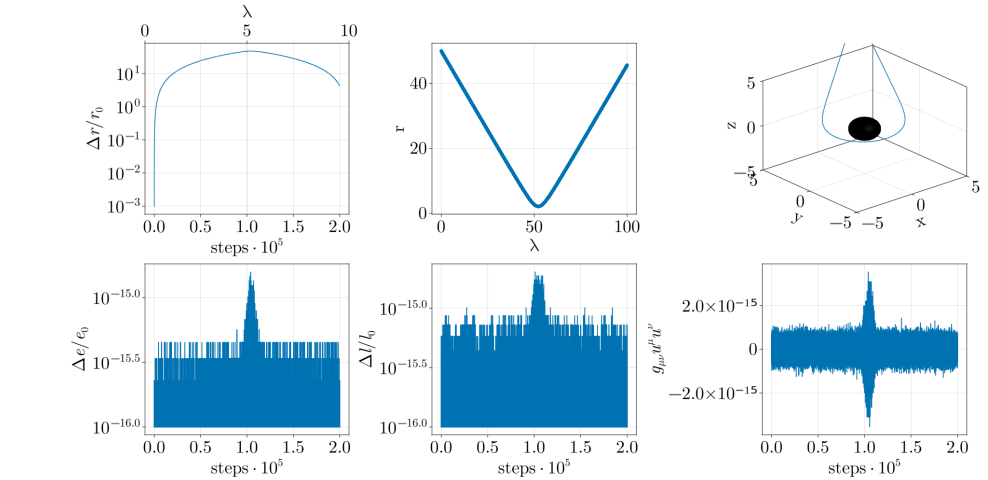


Validation Strategy

- Test on three fundamental orbit classes in near-extremal Kerr ($a = 0.998$):
  1. **Scattering:** $|b| > b_c$ - photon escapes
  2. **Spiral:** $|b| \approx b_c$ - multiple windings (chaotic)
  3. **Capture:** $|b| < b_c$ - falls into horizon
- Monitor: $\Delta E/E_0$, $\Delta L_z/L_{z0}$, and null constraint violation.

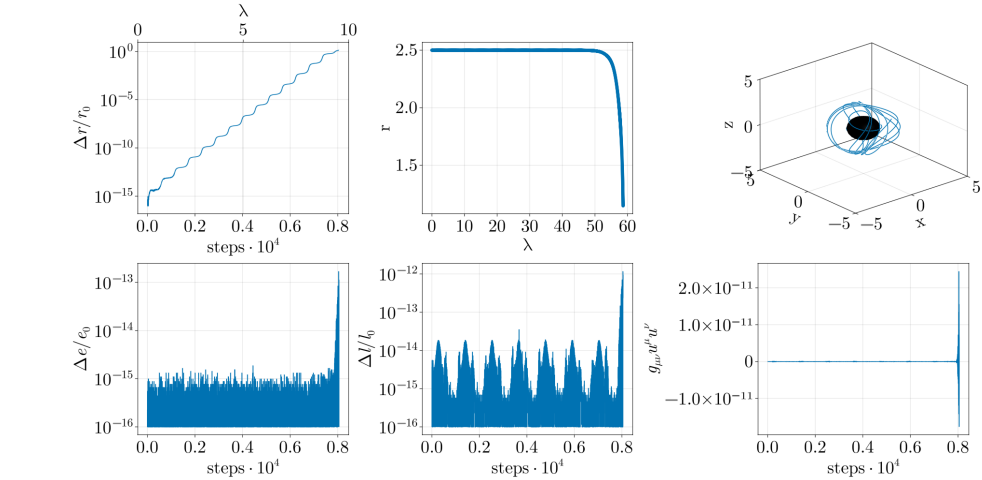
Scattering Orbit

- Photon approaches, reaches r_{min}, escapes ("V" shape in $r(\lambda)$).



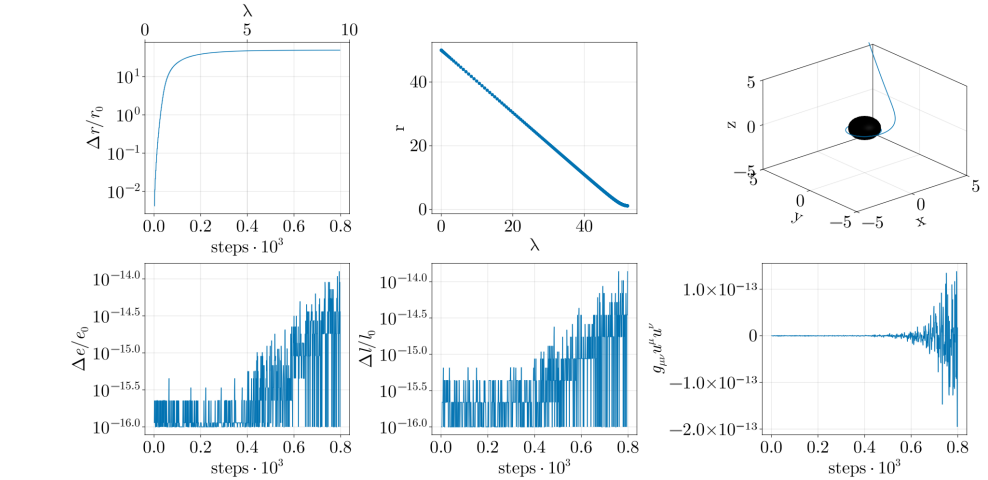
The Stress Test: Unstable Spiral

- Photon orbits near the photon sphere - extreme Lyapunov sensitivity.



Direct Capture

- Photon plunges monotonically into the horizon.



How We Compute the Shadow

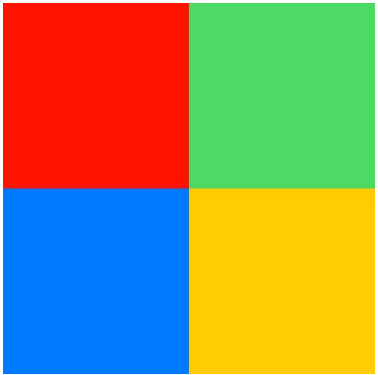
- **Backward Ray Tracing:** We place a virtual camera at distance $r_0 \gg M$ and inclination θ_0.
- **For each pixel:** Trace the photon backwards in time.
  - If it crosses the horizon $\rightarrow$ black pixel (shadow).
  - If it escapes $\rightarrow$ map to background image.

The Shadow Boundary

The shadow edge corresponds to photons that asymptotically approach the unstable photon sphere - the most numerically demanding trajectories.

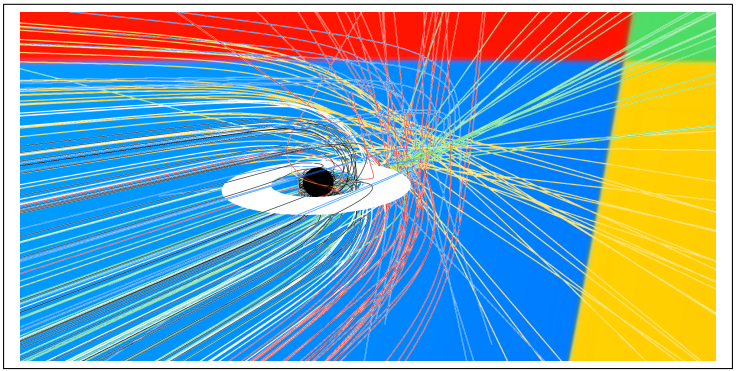
Calibration: The 4-Color Test Image

- **Why calibrate?** Before using realistic backgrounds, we must verify the **UV mapping** and **handedness** of the celestial sphere projection.
- **The test:** A simple equirectangular image with 4 distinct quadrants (red, green, blue, yellow).
- **What we check:** After gravitational lensing, the colors must appear in the physically correct positions - confirming that our coordinate conventions match the spacetime geometry.



Educational Animation: Ray Tracing in Action

- We "shoot" one virtual photon from each pixel of the observer's screen backwards in time.
- These rays travel through the curved spacetime of the black hole, following exact geodesic equations.
- The final destination of each ray (e.g., hitting the accretion disk, falling into the event horizon, or escaping to background stars) dictates the exact color assigned to that pixel.



Calibration: Lensed Result

The same 4-color image, now lensed by a near-extremal Kerr black hole ($a = 0.998$).

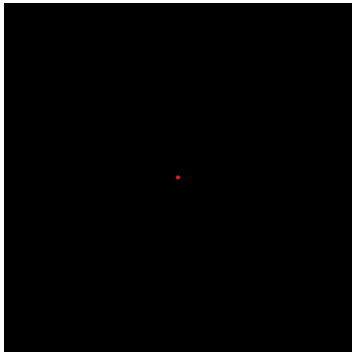
Observations:

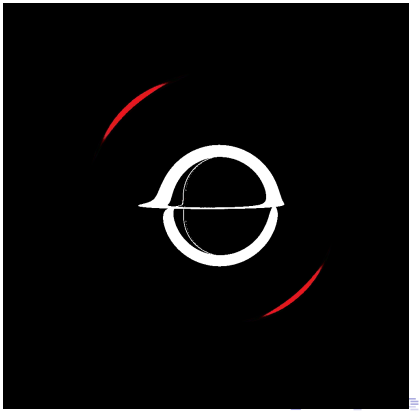
- The central black region is the **shadow** (photons captured by the horizon).
- The white ring is the **photon ring** (photons that orbited the black hole).
- The colors are distorted and wrapped around the shadow, confirming correct handedness.
- The "D" shape asymmetry is clearly visible due to frame-dragging.



Gravitational Lensing: Point Source Test

- **Setup:** A single point source (red dot) placed directly behind the black hole.
- **What we observe:**
  - **Primary & Secondary Arcs:** Photons passing on either side of the black hole ($n = 0, 1$).
  - **Higher-order Sub-rings:** Photons that orbited multiple times ($n \geq 2$), visible as tiny dots near the shadow boundary.





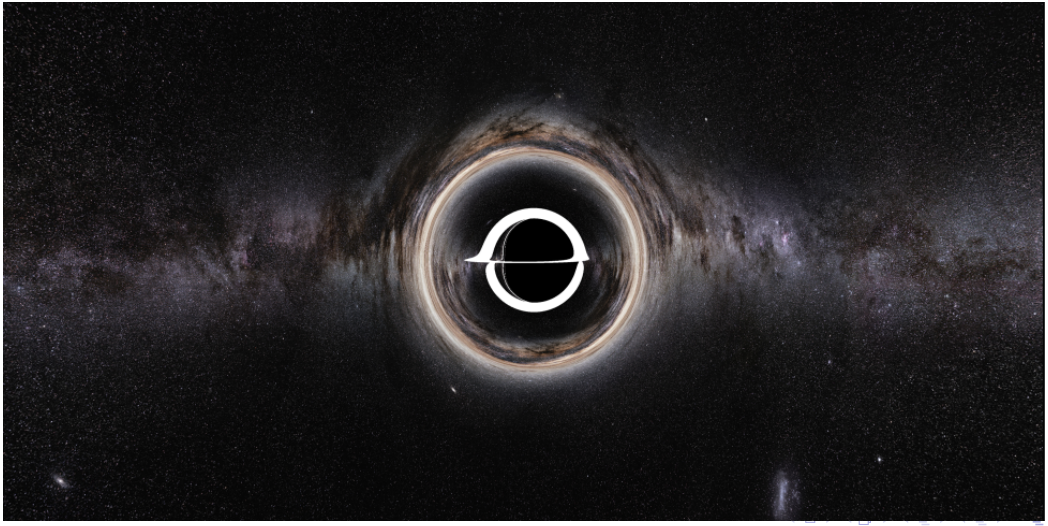
Realistic Imaging: Milky Way Background - Source

- Escaping rays are mapped to an equirectangular Milky Way image.
- Calibration with the 4-color test image ensures correct handedness and physical orientation.



Realistic Imaging: Milky Way Background - Result

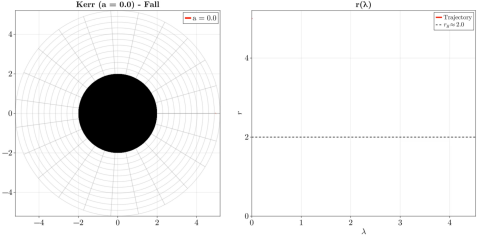
This demonstrates pure gravitational lensing of a realistic background, without local emission.



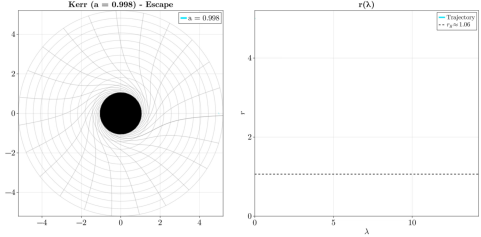
Frame Dragging Evolution

As spin a increases, the spacetime geometry is dragged along the rotation direction.

Schwarzschild ($a = 0$)



Near-Extremal Kerr ($a = 0.998$)



Plasma Model: Thin Accretion Disk

- **Physical setup:** Geometrically thin, optically thick accretion disk in the equatorial plane ($\theta = \pi/2$).
- **Emission profile:** Local emissivity $\epsilon(r)$ follows an exponential density profile, peaking near the Innermost Stable Circular Orbit (ISCO) for each spin parameter..
- **Radiative transfer:** The observed intensity along each ray is:

$$I_{\text{obs}} = \int g^4 \, \epsilon(r) \, d\lambda \tag{9}$$

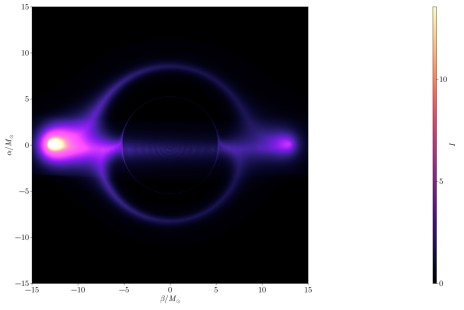
where g is the relativistic redshift factor.

Key Relativistic Effects

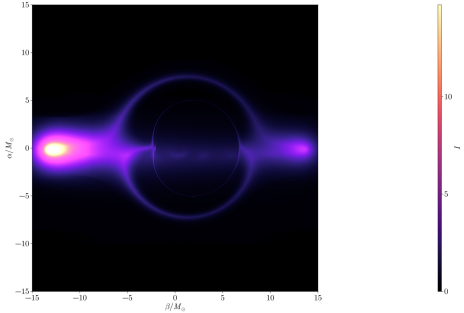
- **Doppler beaming (g^4):** Approaching plasma appears much brighter than receding plasma.
- **Gravitational lensing:** Light from the far side of the disk is bent over and under the black hole.
- **Photon ring:** Multiple windings create a self-similar hierarchy of sub-rings near the shadow boundary.

Shadow Morphology: Spin Dependence

- The mathematical shadow is the region of captured photons.
- **Schwarzschild ($a = 0$):** Perfectly circular shadow due to spherical symmetry.
- **Kerr ($a = 0.998$):** Pronounced "D" shape asymmetry caused by frame-dragging.



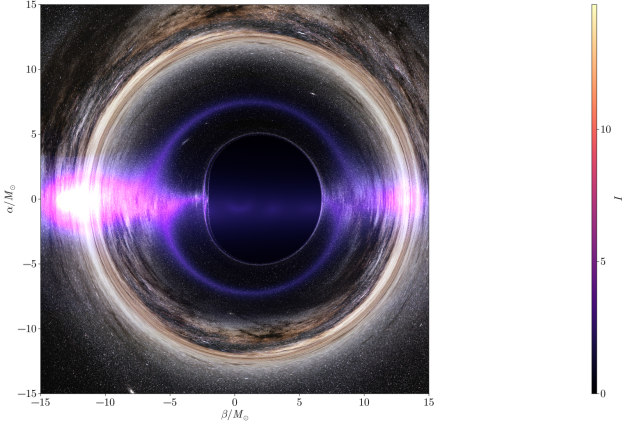
Perfect circle ($a = 0$).



"D" shape from frame-dragging ($a = 0.998$).

Combined Scenario: Accretion Disk + Milky Way Background

- The ultimate test of the framework: combining local astrophysical emission with a realistic celestial background.
- This synthetic image closely mimics the type of data analyzed by the Event Horizon Telescope (EHT).



Conclusions

- Developed a robust, metric-agnostic ray tracing framework in Julia.
- **Automatic Differentiation:** Exact metric derivatives, no truncation error.
- **Manifold Projection:** Invariants conserved to machine precision.
- Successfully resolved higher-order photon sub-rings and complex shadow morphologies.
- **High Performance:** On a standard 16-core laptop, the framework processes computationally demanding trajectories at an average rate of **1,000,000 rays in less than an hour**.

Thank You!

Questions?